

Thank you for your constructive and encouraging feedback, particularly regarding the use of UML artefacts to illustrate both the causal chain and mitigation pathways of SQL injection vulnerabilities.

With regard to the suggestion of explicitly integrating SAST and DAST into the UML flow, it is important to clarify the **intended scope of the model**. The UML diagrams in this contribution are deliberately constrained to a **single, concrete vulnerability (OWASP A03: Injection)** and focus on the application-level design and implementation decisions that lead to exploitability. Incorporating SAST or DAST directly into this vulnerability-specific UML flow would reduce methodological clarity, as these tools operate **across the Secure Software Development Life Cycle (SDLC)** rather than within a single vulnerability path.

That said, **SAST and DAST are fundamental mechanisms within a defence-in-depth Secure SDLC**. SAST is most effective when executed early and frequently, ideally on individual the so called “git-commits”, where code changes are small and context-specific. This aligns with the *shift-left security* principle, which emphasises early detection to reduce remediation cost and risk (McGraw, 2006; NIST, 2022). Running SAST only at merge or release stages often results in delayed feedback and diminished preventive value.

DAST, by contrast, is appropriately positioned in staging or UAT environments, where the application is exercised dynamically. Crucially, such environments should be a **near 1:1 replica of production**, as configuration drift can lead to false negatives and an inaccurate assessment of runtime risk (OWASP, 2023). This distinction reinforces that SAST and DAST complement, rather than replace, secure design and coding practices.

In summary, while SAST and DAST are essential to a mature Secure SDLC, they sit **architecturally outside the UML modelling of a single vulnerability**, which in this case is intentionally used to preserve analytical focus and to demonstrate how insecure design decisions manifest at the application level.

References

- McGraw, G. (2006) *Software Security: Building Security In*. Boston: Addison-Wesley.
- NIST (2022) *Secure Software Development Framework (SSDF)*. NIST SP 800-218. Available at: <https://www.nist.gov/ssdf>
- OWASP (2023) *OWASP Top 10 – A03: Injection*. Available at: <https://owasp.org/Top10/>